

2026년도 전국기능경기대회

직 종 명	클라우드컴퓨팅	과 제 명	Small challenge	과제번호	제 2과제
경기시간	4시간	비 번 호		심사위원 확 인	(인)

1. 요구사항

- 1) EKS Scaling
- 2) VPC Lattice
- 3) Container logging
- 4) REST API Implement

2. 선수 유의사항

※ 다음 유의사항을 고려하여 요구사항을 완성하시오.

- 1) 문제에 제시된 괄호 <>는 변수를 뜻함으로 선수가 적절히 변경하여 사용해야 합니다.
- 2) Security Group의 80/443 outbound는 anyopen하여 사용할 수 있도록 합니다.
- 3) Tag 정보가 맞지 않거나 권한 문제가 생겨 채점이 불가하지 않도록 주의합니다.
- 4) 각 문제에서 요구하는 AWS region에 리소스를 생성하도록 합니다.
- 5) 채점시 cloud shell을 이용해 진행합니다. 브라우저를 통해 접근하며 권한 및 접속 문제가 없도록 주의합니다.
- 6) 과제 지침에 명시되지 않은 서비스는 사용할 수 없습니다.
- 7) 문제에서 주어지지 않은 값들은 AWS Well-Architecture 6 pillars를 기준으로 적절히 값을 설정해야 합니다.

3. EKS Scaling

해당 Module은 ap-northeast-2에서 진행합니다. EKS를 사용하여 Pod 및 Node가 스케일링 되는 시스템을 구성하고자 합니다. 자세한 사항은 아래를 참고하여 구성합니다.

VPC 구성

Subnet Name	CIDR	VPC	AZ
wsc-scaling-sn-pub-a	10.11.0.0/24	wsc-scaling-vpc	ap-northeast-2a
wsc-scaling-sn-pub-c	10.11.1.0/24	wsc-scaling-vpc	ap-northeast-2c
wsc-scaling-sn-priv-a	10.11.10.0/24	wsc-scaling-vpc	ap-northeast-2a
wsc-scaling-sn-priv-c	10.11.11.0/24	wsc-scaling-vpc	ap-northeast-2c

Bastion 구성

EKS 구성을 및 Object 채점을 위하여 Bastion 인스턴스를 생성합니다. 해당 인스턴스는 EKS 클러스터에 접근할 수 있어야 하며, 채점 과정에서 IP가 갑작스럽게 변경되는 상황을 방지하기 위해 재시작 이후에도 동일한 IP를 유지하도록 구성해야 합니다. Public Subnet AZ A에 배포하며 Bastion은 awscli 및 kubectl 입력 시 Admin Policy에 상응하는 권한을 가지고 있어야 합니다.

- Instance Name: wsc-scaling-bastion
- Instance Type: t3.medium

SQS 구성

KEDA를 사용한 이벤트 기반 스케일링 구성을 위해 SQS를 생성합니다. 명시되어 있지 않는 값은 모두 기본값을 사용합니다.

- SQS Name: wsc-scaling-sqs

EKS 구성

Amazon EKS를 사용하여 Container Orchestration 환경을 구성합니다. 해당 EKS는 Managed NodeGroup을 사용합니다.

- EKS Cluster Name: wsc-scaling-cluster
- EKS NodeGroup Instance Type: t3.medium
- EKS NodeGroup Name: wsc-scaling-node
- EKS Node Name: wsc-scaling-node
- EKS NodeGroup Labels: { dedicated: scaling }
- EKS Node Group Min Size: 2
- EKS Node Group Max Size: 10

EKS Object 구성

Amazon EKS 환경에서 애플리케이션을 배포하기 위하여 Deployment를 구성합니다. Deployment는 “busybox:latest” 이미지를 사용하도록 하며, Commnad 부분에 Sleep 명령어를 사용하여 구성합니다. 나머지 값은 선수가 임의로 구성합니다. 모든 Scaling 구성은 “wsc-scaling” Namespace에 논리적으로 구분되어야 합니다.

- Deployment Name: wsc-scaling-deploy
- Deployment Labels: { dedicated: scaling }
- CPU Limit: 500m / CPU Request: 250m
- Memory Limit: 512Mi / Memory Request: 256Mi

Scaling 구성

Amazon EKS 환경에서 KEDA와 Karpenter를 활용한 Scaling 환경을 구성합니다. KEDA는 SQS 메시지 수를 기준으로 Pod를 스케일링하며, 메시지가 없을 경우 최소 2개가 유지 되어야 합니다. SQS 대기열을 30초 주기로 확인하며, 메시지 5개당 Pod 1개가 증가하도록 설정합니다. 또한 Pod 증가로 인해 Node Group의 자원이 부족할 경우 karpenter를 통해 Node를 자동 확장하고, 유휴 상태의 Node는 자동 제거되도록 구성합니다.

- ScaledObject Name: wsc-scaling-scaledobject
- Karpenter Node Pool Limit CPU: 100
- Karpenter Node Pool Limit Memory: 200Gi

4. VPC Lattice

해당 Module는 ap-southeast-1에서 진행합니다. 서로 다른 VPC 간 통신을 VPC Lattice을 사용하여 구성합니다.

Application

Python 3.14에서 개발된 version1.py와 version2.py가 배포됩니다. 애플리케이션 실행 시 TCP 8080 포트로 바인딩됩니다. 애플리케이션은 /healthcheck 경로를 통해 상태 확인을 제공합니다. 제공된 배포파일을 임의로 수정하여 사용하지 않도록 합니다.

Category	Path	Method	Response Body
Version 1	/version	GET	{ "version" : "v1" }
Version 2	/version	GET	{ "version" : "v2" }
Version 1, 2	/healthcheck	GET	{ "status" : "ok" }

VPC 구성

VPC를 사용하여 Cloud Networking을 구성합니다. Name Tag 뒤에 있는 알파벳은 가용영역을 의미하며, 자세한 사항은 아래를 참고하여 구성합니다.

Hub VPC

Name	Type	CIDR	Gateway
wsc-hub-vpc	VPC	10.0.0.0/16	-
wsc-hub-sn-pub-a	Subnet	10.0.0.0/24	Internet Gateway
wsc-hub-sn-pub-c	Subnet	10.0.1.0/24	Internet Gateway

Spoke VPC

Name	Type	CIDR	Gateway
wsc-spoke-vpc	VPC	192.168.0.0/16	-
wsc-spoke-sn-pub-a	Subnet	192.168.0.0/24	Internet Gateway
wsc-spoke-sn-pub-c	Subnet	192.168.1.0/24	Internet Gateway
wsc-spoke-sn-priv-a	Subnet	192.168.2.0/24	NAT Gateway
wsc-spoke-sn-priv-c	Subnet	192.168.3.0/24	NAT Gateway

Bastion 구성

서버 접근을 위해 Bastion 서버를 구성합니다. 해당 서버는 Hub VPC에 Public Subnet AZ A에 구성하도록 하며, 접근 불가할 시 채점이 불가하므로 반드시 SSH를 통한 접속과 권한 문제가 없도록 합니다. 서버가 재시작하여도 Bastion IP가 변동되지 않게 구성합니다. SSH Password 방식을 통해 Bastion에 접근하며, Password는 "Skill53##" 으로 지정합니다.

- EC2 Instance Name: wsc-hub-bastion
- EC2 Instance Type: t3.small

Application 구성

애플리케이션을 배포할 Application 서버를 구성합니다. 해당 서버는 Spoke VPC에 Private Subnet AZ A에 구성하도록 합니다.

- EC2 Instance Name: wsc-spoke-app-v1, wsc-spoke-app-v2
- EC2 Instance Type: t3.medium

Load Balancer 구성

Spoke VPC 내 Private Subnet에 위치하며, Internal ALB 형태로 구성합니다. /healthcheck API 요청 시 403 “Restrict access to api” 를 반환하도록 하며, Application에 명시된 API를 제외한 접근은 404 “Not Found” 를 반환하도록 합니다. 자세한 사항은 아래를 참고하여 구성합니다.

- Load Balancer Name: wsc-spoke-app-alb
- Listen Port: HTTP 80
- Target Group Name: wsc-spoke-v1-tg, wsc-spoke-v2-tg

VPC Lattice 구성

VPC Lattice을 사용하여 Hub VPC와 Spoke VPC간 애플리케이션 통신 환경을 구성합니다. 모든 요청은 반드시 VPC Lattice를 경유하여 전달되어야 하며, 직접적인 VPC Peering 또는 Private IP 기반 접근은 허용하지 않습니다. Hub VPC에서 발생한 요청은 VPC Lattice Service Network를 통해 Spoke VPC 내부의 Internal ALB로 전달되어야 하며, ALB는 각 애플리케이션 버전에 따라 트래픽을 분산하여야 합니다. 또한 VPC Lattice Listener Rule을 통해 HTTP Header 기반 라우팅 정책을 적용합니다. 요청에 version: v1 헤더가 포함된 경우 해당 요청은 반드시 wsc-spoke-v1-tg로 전달되며, version: v2 헤더가 포함된 경우 wsc-spoke-v2-tg로 전달되도록 구성합니다. 해당 Header가 존재하지 않는 일반 요청은 기존 Weighted Routing 정책을 따라 v1 90%, v2 10% 비율로 분산되어야 합니다. 이때 Header 기반 라우팅은 Weighted Routing보다 우선적으로 평가되도록 Rule Priority를 구성합니다. 자세한 사항은 아래를 참고하여 구성합니다.

- VPC Lattice Service Network Name: wsc-app-service-network
- VPC Lattice Service Name: wsc-app-service
- Lattice Target Group:

Name	Priority	Weight
wsc-spoke-v1-tg	10	100
wsc-spoke-v2-tg	20	100

5. Container Logging

해당 Module은 ap-northeast-1에서 진행합니다. EC2 인스턴스에서 Docker 컨테이너로 flask 애플리케이션을 실행하고, EC2에 설치된 Fluent Bit이 컨테이너 로그를 수집하여 EKS에 배포된 Loki로 전송하는 시스템을 구성합니다. 로그 발생부터 Grafana 대시보드 도달까지 최대 10초이내이어야 합니다.

VPC 구성

애플리케이션 서버(EC2)와 로깅 스택(EKS)을 동일한 VPC 내에 구성합니다. EC2는 Public Subnet에, EKS 노드는 Private Subnet에 배포합니다. Subnet 이름 뒤에 영문은 AZ(가용영역)를 뜻합니다.

VPC Name	CIDR
wsc-logging-vpc	10.3.0.0/16

Subnet Name	CIDR	VPC	Gateway
wsc-logging-sn-pub-a	10.3.0.0/24	wsc-logging-vpc	Internet Gateway
wsc-logging-sn-pub-c	10.3.1.0/24	wsc-logging-vpc	Internet Gateway
wsc-logging-sn-priv-a	10.3.2.0/24	wsc-logging-vpc	Nat Gateway
wsc-logging-sn-priv-c	10.3.3.0/24	wsc-logging-vpc	Nat Gateway

EKS 구성

로깅 스택의 효율적인 운영을 위해 EKS 클러스터를 구축합니다. 해당 클러스터는 Loki와 Grafana 배포 전용으로 사용하며, 애플리케이션 워크로드와 분리된 환경으로 구성합니다. Node는 최소 2개 이상 구성하도록 합니다. 추가로 모든 로깅 스택(Loki, Grafana)은 모두 “wsc-logging “ Namespace에서 동작되도록 합니다.

- EKS Cluster Name : wsc-logging-cluster
- EKS Cluster Version : 1.35
- EKS Node Instance Type / AMI : t3.medium / Amazon Linux 2023
- EKS NodeGroup Name : wsc-logging-ng
- EKS Node Name : wsc-logging-node

Loki 구성

Helm을 사용하여 로그 저장소 역할을 하는 Loki를 EKS에 배포합니다. EC2의 Fluent Bit이 VPC 외부(Public -> NLB)를 통해 로그를 Push할 수 있도록 Loki Service를 LoadBalancer(NLB)로 노출합니다.

- namespace : wsc-logging
- Deploy Mode : SingleBinary
- Storage : filesystem(PVC:10Gi)
- Port : 3100
- Helm Release Name: loki

Grafana 구성

Helm을 사용하여 로그 시각화를 위한 Grafana를 EKS에 배포합니다. Loki를 DataSource로 등록하고 아래 4종 패널을 포함하는 대시보드를 구성합니다. 모든 패널은 LogQL을 사용하며 Fluent Bit이 전송한 로그 레이블을 기준으로 조회합니다. loki와 동일하게 Service를 LoadBalancer(NLB)로 노출합니다. 대시보드는 1시간 내 로그를 5초 단위로 새로고침 되어야 합니다.

- namespace : wsc-logging
- Admin User Info : ID : wsc2026-admin-{비번호} / Password : admin{비번호}!
- Dashboard Name : WSC2026 Container Logs
- Helm Release Name: grafana

Dashboard Panel

패널	패널 이름	LogQL 쿼리	시각화 타입
1	Any Log	{namespace="wsc-app-log"}	Logs
2	INFO Log Count	count_over_time({namespace="wsc-app-log"} = "INFO" [1m])	Time Series
3	ERROR Log Count	count_over_time({namespace="wsc-app-log"} = "ERROR" [1m])	Time Series
4	WARNING Log Count	count_over_time({namespace="wsc-app-log"} = "WARNING" [1m])	Time Series

EC2 애플리케이션 및 Fluent Bit 구성

제공된 배포파일(app.py, requirements.txt, Dockerfile)을 EC2에 app/ 경로에 업로드하고, Docker 이미지를 빌드한 뒤 컨테이너를 실행합니다. 컨테이너는 항상 재시작되도록 설정하며, 로그는 Docker 기본 json-file 드라이버로 기록되어야 합니다. 또한 Fluent Bit을 EC2 호스트에 직접 설치하여 systemd 서비스로 실행합니다. Fluent Bit은 Docker 컨테이너 로그 경로를 감시하고 record_modifier 필터로 namespace 레이블 값을 wsc-app-log로 추가한 뒤 Loki 엔드포인트로 전송해야 합니다. Instance Server는 1개로 동작해야 하며, 제공되는 모든 배포파일은 수정할 수 없습니다.

- EC2 Instance Name : wsc-logging-app-bastion
- EC2 Instance Type / AMI : t3.small / Amazon Linux 2023
- Container Name : wsc-log-app
- Application Port :TCP_5000

Path	Method	Description
/	GET	서비스 상태 확인
/health	GET	헬스체크 (200 OK 반환)
/generate?count=<N>	GET	N건의 랜덤 로그 발생 (INFO / WARNING / ERROR 혼합)
/error	GET	ERROR 레벨 로그 1건 강제 발생

- Log Time Format : %Y-%m-%dT%H:%M:%S.%L(Asia/Seoul Timezone을 설정해야 합니다.)

6. REST API Implement

해당 Module은 us-east-1에서 진행합니다. API Gateway, Lambda, DynamoDB를 사용하여 Serverless Architecture를 구성합니다. 자세한 사항은 아래를 참고하여 구성합니다.

API Description

아래 표를 참고하여 소스 코드를 개발하도록 합니다. 단, /healthcheck에 관련된 API는 Lambda에서 개발하지 않도록 합니다. 또한 동일 사용자 데이터는 중복 저장이 허용되지 않아야 하며, 동일 요청이 여러 번 호출되더라도 데이터 정합성이 유지되어야 합니다. Retry 상황(Lambda Retry, API Gateway Retry, Client Retry 포함)에서도 중복 데이터가 저장되면 {'message': 'User already exists'}을 반환합니다. 입력 데이터에 대한 Validation이 반드시 수행되어야 하며, 잘못된 요청은 Lambda까지 전달되지 않아야 합니다. 존재하지 않는 데이터 요청 시 {'message': 'User not found'}를 반환해야 하며, 잘못된 Query String 요청 시 {"message": "Missing required request parameters: [age]"}를 반환합니다.

Path	Method	Description
/user	POST	Request Body
		{ "name": "kim", "age": 19, "country": "korea" }
		Response Body
	GET	{ "message": "User created successfully" }
		Query String
		?name=kim&age=19
		Response Body
		{ "name": "kim", "age": 19, "country": "korea" }

API Gateway 구성

API Gateway는 REST API를 제공하는 API 서비스를 구축합니다. 자세한 사항은 아래를 참고하여 구성합니다. 또한 /v1/user API 호출은 API Key 인증을 사용하여 처리되어야 하며, API Key 없이 호출된 요청은 API Gateway 단계에서 차단되어야 합니다. 잘못된 요청은 Lambda까지 전달되지 않아야 합니다.

- API Gateway Name: wsc-rest-api
- API Gateway Stage Name: prod
- API Key Name: wsc-rest-api-key
- API Deployment Stage:

Path	Response Body	Description
/v1/user	API 설명 참고	API 설명을 참고하여 애플리케이션에 대한 API 구성
/v1/healthcheck	{ "status": "ok" }	JSON 형식으로 출력되어야 하며, MOCK Intergration Type을 사용하여 구성

Lambda 구성

요구 사항 및 API 설명을 참고하여 Lambda Function을 생성 및 개발합니다. Exception 발생 시 Stack Trace가 외부에 노출되어서는 안 됩니다. boto3 Client 재사용 및 Connection Reuse를 고려해야 합니다. Retry-safe 구조를 고려하여 중복 요청 처리 및 데이터 정합성 보장이 가능해야 합니다.

- Lambda Name: wsc-rest-function
- Runtime: Python 3.14

DynamoDB 구성

Lambda의 데이터를 저장하기 위해 NoSQL 기반 데이터베이스인 DynamoDB를 구성합니다. 동일 사용자 데이터 중복 저장 방지를 위해 Conditional Write 기반으로 데이터 정합성을 보장해야 합니다. 또한 3000 RPS 이상의 Burst Traffic 환경에서도 안정적으로 동작할 수 있도록 설계해야 하며, Hot Partition 발생 가능성을 고려한 구조로 구성해야 합니다.

- Table Name: wsc-rest-table
- Description of table

Key Name	Data Type	ETC
name	String	Partition Key
age	Number	
country	String	